

MÉTODOS DE OPTIMIZACIÓN TAREA 5

Mariana Belén Cruz Rodríguez

today

Problema 3: Ramificación y Acotamiento

Use ramificación y acotamiento

$$\max z = 10x_1 + 9x_2 + 7x_3$$

$$4x_1 + 5x_2 + 3x_3 \leq 23$$

$$3x_1 + 2x_2 + 4x_3 \leq 20$$

$$x_1, x_2, x_3 \in \mathbb{Z}_{\geq 0}$$

Solución

Para esto primero resolvemos de forma continua para a partir de esto ramificar y acotar.

```
1 library(lpSolve)
2
3 #funci n objetivo
4 obj <- c(10, 9, 7)
5
6 #restricciones
7 A <- matrix(c(
8 4,5,3,
9 3,2,4
10 ), nrow = 2, byrow = TRUE)
11
12 #RHS
13 b <- c(23,20)
14 dir <- c("<=", "<=")
15
16 #modelo
17 modelo <- lp("max", obj, A, dir, b)
18
19 modelo$solution
20 modelo$objval
```

```
> modelo$solution
[1] 5.75 0.00 0.00
> modelo$objval
[1] 57.5
```

Se tiene que:

$$x_1 = 5.75, \quad x_2 = 0, \quad x_3 = 0, \quad Z = 57.5$$

Como x_1 no es entera, ramificamos:

Vamos a ramificar en

$$x_1 \leq 5 \quad \text{y} \quad x_1 \geq 6$$

Nodo 1: $x_1 \leq 5$

```
1 A1 <- rbind(A, c(1,0,0))
2 b1 <- c(23,20,5)
3 dir1 <- c("<=", "<=", "<=")
4
5 n1 <- lp("max", obj, A1, dir1, b1)
6
7 n1$solution
8 n1$objval
```

```
> n1$solution
[1] 5 0 1
> n1$objval
[1] 57
```

Resultado:

$$x_1 = 5, \quad x_2 = 0, \quad x_3 = 1, \quad Z = 57$$

Esta solución es entera y factible.

Nodo 2: $x_1 \geq 6$

```
1 A2 <- rbind(A, c(1,0,0))
2 b2 <- c(23,20,6)
3 dir2 <- c("<=", "<=", ">=")
4
5 n2 <- lp("max", obj, A2, dir2, b2)
```

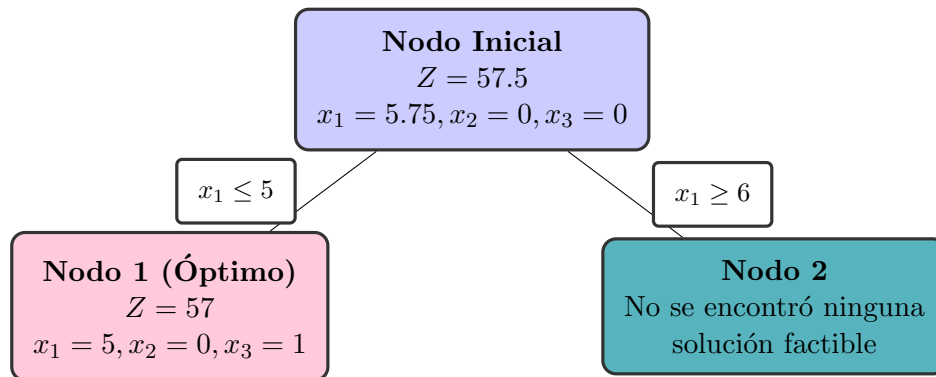
```
> n1$solution
[1] 0 0 0
> n1$objval
[1] 0
```

No hay soluciones factibles, además se verifica que:

$$4(6) = 24 > 23 \Rightarrow \text{No factible}$$

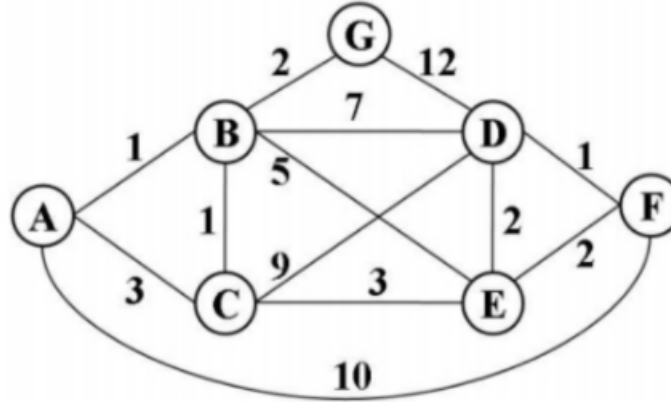
Como ya no hay nada más para ramificar se tiene entonces que la solución óptima al PLE (al ser la única) es

$$x^* = (5, 0, 1), \quad Z^* = 57$$



Problema 8: Árbol de Expansión Mínima

Use el algoritmo de expansión mínima en el grafo desde el punto C .



Solución

Consideramos la matriz de adyacencia ponderada correspondiente al grafo:

$$\begin{array}{c}
 \begin{matrix} & A & B & C & D & E & F & G \end{matrix} \\
 \begin{matrix} A \\ B \\ C \\ D \\ E \\ F \\ G \end{matrix} \begin{pmatrix}
 0 & 1 & 3 & 0 & 0 & 10 & 0 \\
 1 & 0 & 1 & 7 & 5 & 0 & 2 \\
 3 & 1 & 0 & 9 & 3 & 0 & 0 \\
 0 & 7 & 9 & 0 & 2 & 1 & 12 \\
 0 & 5 & 3 & 2 & 0 & 2 & 0 \\
 10 & 0 & 0 & 1 & 2 & 0 & 0 \\
 0 & 2 & 0 & 12 & 0 & 0 & 0
 \end{pmatrix}
 \end{array}$$

Sea $N = \{A, B, C, D, E, F, G\}$ el conjunto de los nodos de la red.

Y sean:

C_k : El conjunto de los nodos que se han conectado en forma permanente en la iteración k .

$\bar{C}_k$: El conjunto de los nodos que todavía se deben conectar en forma permanente.

Al empezar se tiene que $C_0 = \emptyset$ y $\bar{C}_0 = N$.

PASO 1:

Se tiene que $C_1 = \{C\}$ y $\bar{C}_1 = \{A, B, D, E, F, G\}$, pues el problema pide iniciar desde el nodo C .

PASO 2:

Buscamos el nodo $j^* \in \bar{C}_1$ que tenga el menor costo de ir a C . Como 1 es el costo más bajo positivo de la fila de C en la matriz de adyacencia, que corresponde al nodo B , entonces conectamos a C con B , y así:

$$C_2 = \{C, B\} \quad \text{y} \quad \bar{C}_2 = \{A, D, E, F, G\}$$

PASO 3:

Se escoge $j^* \in \bar{C}_2$ que produzca el menor costo de ir a un nodo de C_2 . Al observar las opciones desde C y B , el costo más bajo es 1, que corresponde conectar B y A . Con lo cual:

$$C_3 = \{A, B, C\} \quad \text{y} \quad \bar{C}_3 = \{D, E, F, G\}$$

PASO 4:

Se busca el menor costo desde $\{A, B, C\}$ hacia $\bar{C}_3$. El costo más bajo es 2, que corresponde al enlazar B y G , así:

$$C_4 = \{A, B, C, G\} \quad \text{y} \quad \bar{C}_4 = \{D, E, F\}$$

PASO 5:

Se busca el menor costo desde $\{A, B, C, G\}$ hacia $\bar{C}_4$. El costo más bajo es **3**, que corresponde al enlace entre C y E :

$$C_5 = \{A, B, C, E, G\} \quad \text{y} \quad \bar{C}_5 = \{D, F\}$$

PASO 6:

Se busca el menor costo desde C_5 hacia $\bar{C}_5$. El costo más bajo es **2**, correspondiente a enlazar E y D :

$$C_6 = \{A, B, C, D, E, G\} \quad \text{y} \quad \bar{C}_6 = \{F\}$$

PASO 7:

Se busca el menor costo para conectar el último nodo F . El costo más bajo es **1**, que corresponde al enlace entre D y F :

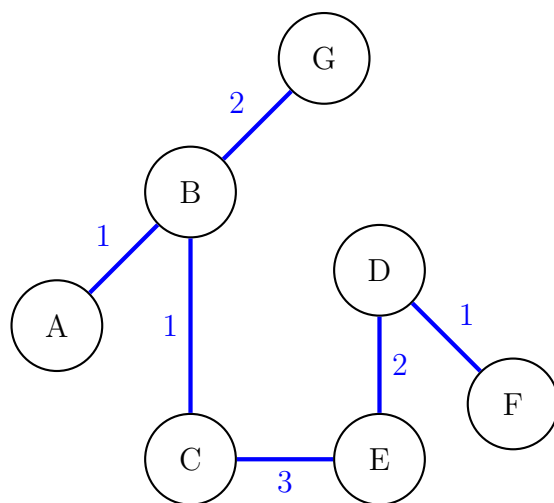
$$C_7 = \{A, B, C, D, E, F, G\} \quad \text{y} \quad \bar{C}_7 = \emptyset$$

PASO 8:

Como $\bar{C}_7 = \emptyset$, el algoritmo termina.

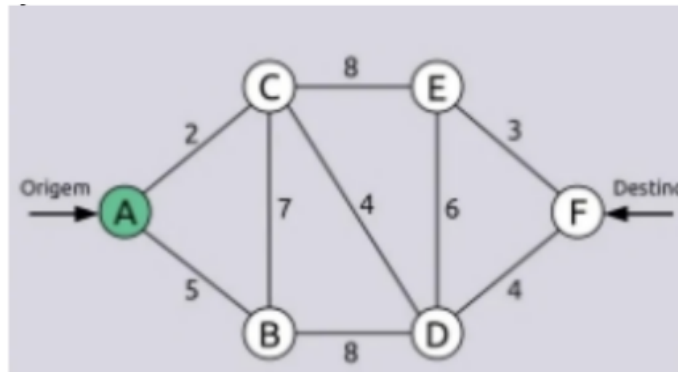
De este modo el árbol de expansión mínima tiene un costo total de:

$$Z^* = 1 + 1 + 2 + 3 + 2 + 1 = 10$$



Problema 13: Algoritmo de Dijkstra

Use el algoritmo de Dijkstra cuando el nodo fuente es A hacia el nodo terminal F.



Solución

Para el nodo inicial que será A, consideramos que tiene etiqueta permanente $[0, --]$ pues es el nodo inicial y no tiene predecesor.

Iteración 1 ($i = A$)

Se puede llegar a los nodos C y B desde el nodo A. Así, la lista de los nodos etiquetados es la siguiente:

Nodo	Etiqueta	Estado
A	$[0, --]$	Permanente
C	$[0 + 2, A] = [2, A]$	Temporal
B	$[0 + 5, A] = [5, A]$	Temporal

Para las etiquetas temporales, el nodo C produce la menor distancia ($u_C = 2$). Entonces, se cambia el estado del nodo C a permanente.

Iteración 2 ($i = C$)

Del nodo C se puede ir a los nodos B, D y E. La lista de nodos etiquetados es:

Nodo	Etiqueta	Estado
A	$[0, --]$	Permanente
C	$[2, A]$	Permanente
B	$[5, A]$	Temporal
D	$[2 + 4, C] = [6, C]$	Temporal
E	$[2 + 8, C] = [10, C]$	Temporal

La etiqueta temporal de B se mantiene como $[5, A]$ ya que $5 < 2 + 7$. El nodo B produce la menor distancia ($u_B = 5$), por lo que su estado cambia a permanente.

Iteración 3 ($i = B$)

Del nodo B se puede ir al nodo D . La lista de nodos es:

Nodo	Etiqueta	Estado
A, C, B	(Varios)	Permanente
D	$[6, C]$	Temporal
E	$[10, C]$	Temporal

La etiqueta de D se mantiene $[6, C]$ pues $6 < 5 + 8$. El nodo D pasa a ser permanente ($u_D = 6$).

Iteración 4 ($i = D$)

Del nodo D se puede ir a E y F . La lista actualizada es:

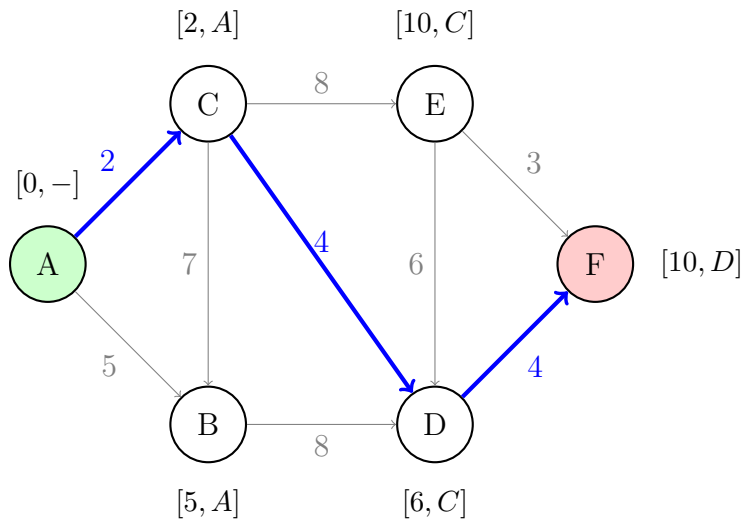
Nodo	Etiqueta	Estado
A, C, B, D	(Varios)	Permanente
E	$[10, C]$	Temporal
F	$[6 + 4, D] = [10, D]$	Temporal

Como el nodo F es el terminal y su distancia es mínima entre las temporales, su estado se vuelve permanente y el algoritmo termina.

La ruta se determina retrocediendo por las etiquetas permanentes:

$$(F) \rightarrow [10, D] \rightarrow (D) \rightarrow [6, C] \rightarrow (C) \rightarrow [2, A] \rightarrow (A)$$

Por lo anterior, la ruta buscada es $\mathbf{A} \rightarrow \mathbf{C} \rightarrow \mathbf{D} \rightarrow \mathbf{F}$, con una longitud total de **10**.



Podemos comprobar esto:

```
1 library(igraph)
2
3 datos <- data.frame(
4   from = c("A", "A", "C", "C", "C", "B", "E", "D", "D"),
5   to   = c("C", "B", "E", "D", "B", "D", "F", "F", "E"),
6   weight = c(2, 5, 8, 4, 7, 8, 3, 4, 6)
7 )
8
9 g <- graph_from_data_frame(datos, directed=FALSE)
10
11
12 #resultado de la secuencia de nodos
13 print(ruta$vpath)
```

```
> # resultado de la secuencia de nodos
> print(ruta$vpath)
[[1]]
+ 4/6 vertices, named, from 2898d14:
[1] A C D F
```

Problema 18: Algoritmo de Floyd

Aplice el algoritmo de Floyd. Genere en cada iteración las matrices de distancias D_i y de Secuencias de nodos S_i .

$$D_0 = \begin{pmatrix} 0 & 3 & \infty & 12 & \infty & \infty \\ 3 & 0 & 4 & \infty & 10 & \infty \\ \infty & 4 & 0 & 2 & \infty & 15 \\ 12 & \infty & 2 & 0 & 5 & \infty \\ \infty & 10 & \infty & 5 & 0 & 3 \\ \infty & \infty & 15 & \infty & 3 & 0 \end{pmatrix}$$

Para resolver este problema, aplicamos la **operación triple**. En cada iteración k , el algoritmo verifica si pasar por el nodo k ofrece una ruta más corta entre cualquier par de nodos (i, j) que la distancia conocida hasta ese momento.

La regla de actualización es:

1. Si $d_{ik} + d_{kj} < d_{ij}$, entonces $d_{ij} = d_{ik} + d_{kj}$.
2. Si ocurre la mejora, se actualiza la matriz de secuencias: $s_{ij} = k$

Iteración 0: Estado Inicial

Se definen las matrices de partida basadas en el grafo original. En S_0 , los valores representan el nodo destino directo.

$$D_0 = \begin{pmatrix} 0 & 3 & \infty & 12 & \infty & \infty \\ 3 & 0 & 4 & \infty & 10 & \infty \\ \infty & 4 & 0 & 2 & \infty & 15 \\ 12 & \infty & 2 & 0 & 5 & \infty \\ \infty & 10 & \infty & 5 & 0 & 3 \\ \infty & \infty & 15 & \infty & 3 & 0 \end{pmatrix}, \quad S_0 = \begin{pmatrix} 0 & 2 & 3 & 4 & 5 & 6 \\ 1 & 0 & 3 & 4 & 5 & 6 \\ 1 & 2 & 0 & 4 & 5 & 6 \\ 1 & 2 & 3 & 0 & 5 & 6 \\ 1 & 2 & 3 & 4 & 0 & 6 \\ 1 & 2 & 3 & 4 & 5 & 0 \end{pmatrix}$$

Iteración 1 ($k = 1$)

Se busca si pasar por el nodo 1 acorta distancias.

- **Mejora (2,4):** $d_{21} + d_{14} = 3 + 12 = 15 < \infty$. Se actualiza $d_{24} = 15$ y $s_{24} = 1$.
- **Mejora (4,2):** Por simetría, $d_{42} = 15$ y $s_{42} = 1$.

$$D_1 = \begin{pmatrix} 0 & 3 & \infty & 12 & \infty & \infty \\ 3 & 0 & 4 & 15 & 10 & \infty \\ \infty & 4 & 0 & 2 & \infty & 15 \\ 12 & 15 & 2 & 0 & 5 & \infty \\ \infty & 10 & \infty & 5 & 0 & 3 \\ \infty & \infty & 15 & \infty & 3 & 0 \end{pmatrix}, \quad S_1 = \begin{pmatrix} 0 & 2 & 3 & 4 & 5 & 6 \\ 1 & 0 & 3 & 1 & 5 & 6 \\ 1 & 2 & 0 & 4 & 5 & 6 \\ 1 & 1 & 3 & 0 & 5 & 6 \\ 1 & 2 & 3 & 4 & 0 & 6 \\ 1 & 2 & 3 & 4 & 5 & 0 \end{pmatrix}$$

Iteración 2 ($k = 2$)

Se producen mejoras al conectar nodos a través del nodo 2.

- **Mejora (1,3):** $d_{12} + d_{23} = 3 + 4 = 7 < \infty \rightarrow s_{13} = 2$.
- **Mejora (1,5):** $d_{12} + d_{25} = 3 + 10 = 13 < \infty \rightarrow s_{15} = 2$.

$$D_2 = \begin{pmatrix} 0 & 3 & 7 & 12 & 13 & \infty \\ 3 & 0 & 4 & 15 & 10 & \infty \\ 7 & 4 & 0 & 2 & 14 & 15 \\ 12 & 15 & 2 & 0 & 5 & \infty \\ 13 & 10 & 14 & 5 & 0 & 3 \\ \infty & \infty & 15 & \infty & 3 & 0 \end{pmatrix}, \quad S_2 = \begin{pmatrix} 0 & 2 & 2 & 4 & 2 & 6 \\ 1 & 0 & 3 & 1 & 5 & 6 \\ 2 & 2 & 0 & 4 & 2 & 6 \\ 1 & 1 & 3 & 0 & 5 & 6 \\ 2 & 2 & 2 & 4 & 0 & 6 \\ 1 & 2 & 3 & 4 & 5 & 0 \end{pmatrix}$$

Iteración 3 ($k = 3$)

Se utiliza el nodo 3 como intermediario.

- **Mejora (1,4):** $d_{13} + d_{34} = 7 + 2 = 9 < 12 \rightarrow s_{14} = 3$.
- **Mejora (2,4):** $d_{23} + d_{34} = 4 + 2 = 6 < 15 \rightarrow s_{24} = 3$.

$$D_3 = \begin{pmatrix} 0 & 3 & 7 & 9 & 13 & 22 \\ 3 & 0 & 4 & 6 & 10 & 19 \\ 7 & 4 & 0 & 2 & 14 & 15 \\ 9 & 6 & 2 & 0 & 5 & 17 \\ 13 & 10 & 14 & 5 & 0 & 3 \\ 22 & 19 & 15 & 17 & 3 & 0 \end{pmatrix}, \quad S_3 = \begin{pmatrix} 0 & 2 & 2 & 3 & 2 & 3 \\ 1 & 0 & 3 & 3 & 5 & 3 \\ 2 & 2 & 0 & 4 & 2 & 6 \\ 3 & 3 & 3 & 0 & 5 & 3 \\ 2 & 2 & 2 & 4 & 0 & 6 \\ 3 & 3 & 3 & 3 & 5 & 0 \end{pmatrix}$$

Iteración 5 ($k = 5$)

Esta iteración genera las mejoras más significativas hacia el nodo terminal 6.

- **Mejora (1,6):** $d_{15} + d_{56} = 13 + 3 = 16 < 22 \rightarrow s_{16} = 5$.
- **Mejora (2,6):** $d_{25} + d_{56} = 10 + 3 = 13 < 19 \rightarrow s_{26} = 5$.

$$D_5 = \begin{pmatrix} 0 & 3 & 7 & 9 & 13 & 16 \\ 3 & 0 & 4 & 6 & 10 & 13 \\ 7 & 4 & 0 & 2 & 7 & 10 \\ 9 & 6 & 2 & 0 & 5 & 8 \\ 13 & 10 & 7 & 5 & 0 & 3 \\ 16 & 13 & 10 & 8 & 3 & 0 \end{pmatrix}, \quad S_5 = \begin{pmatrix} 0 & 2 & 2 & 3 & 2 & 5 \\ 1 & 0 & 3 & 3 & 5 & 5 \\ 2 & 2 & 0 & 4 & 4 & 5 \\ 3 & 3 & 3 & 0 & 5 & 5 \\ 2 & 2 & 4 & 4 & 0 & 6 \\ 5 & 5 & 5 & 5 & 5 & 0 \end{pmatrix}$$

Resultado Final

Como ya se hicieron las 6 iteraciones correspondientes para los 6 nodos, el algoritmo termina. Las matrices finales D_6 y S_6 dan la información necesaria para encontrar la ruta más corta entre cualquier par de nodos.

$$D^* = \begin{pmatrix} 0 & 3 & 7 & 9 & 13 & 16 \\ 3 & 0 & 4 & 6 & 10 & 13 \\ 7 & 4 & 0 & 2 & 7 & 10 \\ 9 & 6 & 2 & 0 & 5 & 8 \\ 13 & 10 & 7 & 5 & 0 & 3 \\ 16 & 13 & 10 & 8 & 3 & 0 \end{pmatrix}, \quad S^* = \begin{pmatrix} 0 & 2 & 2 & 3 & 2 & 5 \\ 1 & 0 & 3 & 3 & 5 & 5 \\ 2 & 2 & 0 & 4 & 4 & 5 \\ 3 & 3 & 3 & 0 & 5 & 5 \\ 2 & 2 & 4 & 4 & 0 & 6 \\ 5 & 5 & 5 & 5 & 5 & 0 \end{pmatrix}$$

Podemos comprobar usando:

```

1
2 # Definir la matriz de distancias inicial D0
3 # Usamos 1000 para representar el infinito (Inf)
4 inf <- 1000
5 D <- matrix(c(0, 3, inf, 12, inf, inf,
6               3, 0, 4, inf, 10, inf,
7               inf, 4, 0, 2, inf, 15,
8               12, inf, 2, 0, 5, inf,
9               inf, 10, inf, 5, 0, 3,
10              inf, inf, 15, inf, 3, 0),
11             nrow = 6, byrow = TRUE)
12
13 # Definir la matriz de secuencias inicial S0
14 S <- matrix(rep(1:6, each = 6), nrow = 6, byrow = FALSE)
15 diag(S) <- 0 # La diagonal no tiene secuencia intermedia
16              inicial
17
18 # Implementaci3n del Algoritmo de Floyd (Operaci3n Triple)
19 n <- nrow(D)
20 for (k in 1:n) {
21   for (i in 1:n) {
22     for (j in 1:n) {
23       # Verificamos la condici3n: d_ik + d_kj < d_ij
24       if (D[i,k] + D[k,j] < D[i,j]) {
25         D[i,j] <- D[i,k] + D[k,j] # Actualizamos costo
26         S[i,j] <- k               # Registramos nodo intermedio
27       }
28     }
29   }
30 }
31
32 # Mostrar resultados finales
33 cat("Matriz de Distancias Final (D6):\n")
34 print(D)
35 cat("\nMatriz de Secuencias Final (S6):\n")
36 print(S)

```

```

Matriz de Distancias Final (D6):
> print(D)
      [,1] [,2] [,3] [,4] [,5] [,6]
[1,]    0    3    7    9   13   16
[2,]    3    0    4    6   10   13
[3,]    7    4    0    2    7   10
[4,]    9    6    2    0    5    8
[5,]   13   10    7    5    0    3
[6,]   16   13   10    8    3    0
> cat("\nMatriz de Secuencias Final (S6):\n")

Matriz de Secuencias Final (S6):
> print(S)
      [,1] [,2] [,3] [,4] [,5] [,6]
[1,]    0    2    2    3    2    5
[2,]    1    0    3    3    5    5
[3,]    2    2    0    4    4    5
[4,]    3    3    3    0    5    5
[5,]    2    2    4    4    0    6
[6,]    5    5    5    5    5    0
>

```

Problema 23: Flujo Máximo

Problema: Flujo máximo

Se desea encontrar el flujo máximo desde el nodo fuente $s = 1$ hasta el sumidero $t = 6$.

Arcos y capacidades

$$(1, 2) = 11, \quad (1, 3) = 9$$

$$(2, 4) = 7, \quad (2, 5) = 4$$

$$(3, 2) = 3, \quad (3, 5) = 8$$

$$(4, 3) = 2, \quad (4, 6) = 6$$

$$(5, 4) = 5, \quad (5, 6) = 7$$

Aplicación del algoritmo de Ford-Fulkerson

Iteración 1 Camino aumentante:

$$1 \rightarrow 2 \rightarrow 4 \rightarrow 6$$

Capacidad mínima:

$$\min(11, 7, 6) = 6$$

Flujo enviado: 6

Iteración 2 Camino:

$$1 \rightarrow 3 \rightarrow 5 \rightarrow 6$$

Capacidad mínima:

$$\min(9, 8, 7) = 7$$

Flujo enviado: 7

Iteración 3 Camino:

$$1 \rightarrow 2 \rightarrow 5 \rightarrow 4 \rightarrow 6$$

Capacidades residuales:

$$(1, 2) = 5, \quad (2, 5) = 4, \quad (5, 4) = 5, \quad (4, 6) = 0$$

No es posible usar completamente este camino (arco saturado).

Se intenta:

$$1 \rightarrow 2 \rightarrow 5 \rightarrow 6$$

Capacidad mínima:

$$\min(5, 4, 0) = 0$$

No aporta flujo.

Iteración 4 Camino:

$$1 \rightarrow 3 \rightarrow 2 \rightarrow 4 \rightarrow 6$$

Capacidades residuales:

$$(1, 3) = 2, (3, 2) = 3, (2, 4) = 1, (4, 6) = 0$$

No se puede aumentar flujo.

Flujo máximo

Sumando los flujos enviados:

$$f_{\max} = 6 + 7 = 13$$

Conclusión

El flujo máximo del nodo 1 al nodo 6 es:

13

```
1 library(igraph)
2
3 g <- graph_from_data_frame(
4   data.frame(
5     from=c(1,1,2,2,3,3,4,4,5,5),
6     to=c(2,3,4,5,2,5,3,6,4,6),
7     capacity=c(11,9,7,4,3,8,2,6,5,7)
8   ),
9   directed=TRUE
10 )
11
12 max_flow(g, source=1, target=6)
```

```

> max_flow(g, source=1, target=6)
$value
[1] 13

$flow
[1] 10  3  7  4  1  3  1  6  0  7

$cut
+ 2/10 edges from 7092c31 (vertex names):
[1] 4->6 5->6

$partition1
+ 5/6 vertices, named, from 7092c31:
[1] 1 2 3 4 5

$partition2
+ 1/6 vertex, named, from 7092c31:
[1] 6

$stats
$stats$nopush
[1] 6

$stats$norelabel
[1] 5

$stats$nogap
[1] 3

$stats$nogapnodes
[1] 1

$stats$nobfs
[1] 1

```